Smart Lecture Q@A System from Video Content

Sinh viên:
23521101 Trần Minh Nhất
23521163 Lê Hồng Phong
23521490 Vũ Hiếu Thiên

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN - ĐẠI HỌC QUỐC GIA TP.HCM
KHOA KHOA HỌC MÁY TÍNH
CS431.Q11

Ngày 17 tháng 2 năm 2026

Mục lục

1. Giới thiệu Hệ thống
2. Xử lý Video và ASR
3. Xử lý Transcript
4. Retrieval Augmented Generation
5. Dữ liệu và Kết quả thực nghiệm
6. Kết luận

Giới thiệu Hệ thống

Bài toán đặt ra

- Các video bài giảng học phần CS431 thường có thời lượng lớn, gây khó khăn cho sinh viên trong việc tra cứu và định vị nhanh các đoạn nội dung chứa lời giải đáp cho các thắc mắc cụ thể.
- Phương pháp tìm kiếm thủ công kém hiệu quả và tiêu tốn thời gian.

Mục tiêu hệ thống

- Phát triển một hệ thống Hỏi – Đáp (Q&A) thông minh tự động, được xây dựng dựa trên nội dung video bài giảng môn CS431.
- Cung cấp cho sinh viên khả năng đặt câu hỏi tự nhiên và nhận được câu trả lời chính xác kèm theo tham chiếu đến đoạn video có liên quan.

Trích xuất Dữ liệu Âm thanh

Mục đích

- Tiền xử lý và chuẩn bị dữ liệu âm thanh đầu vào cho quá trình Nhận dạng Giọng nói Tự động (Automatic Speech Recognition - ASR).

Quy trình xử lý

- Sử dụng công cụ `ffmpeg` để tách kênh âm thanh từ các file video bài giảng.
- Chuẩn hóa âm thanh đầu ra về các thông số kỹ thuật phù hợp với mô hình ASR, ví dụ: tần số lấy mẫu 16kHz, định dạng mono, nhằm tối ưu hóa chất lượng đầu vào.
- Lưu trữ file âm thanh đã xử lý theo cấu trúc thư mục được phân loại theo từng chương hoặc chủ đề.

Chuyển đổi Giọng nói thành Văn bản (ASR)

Mục đích

- Chuyển đổi âm thanh bài giảng thành transcript có timestamp
- Cung cấp dữ liệu đầu vào cho pipeline xử lý văn bản

Phương pháp

- Sử dụng mô hình Whisper để chuyển đổi speech-to-text
- Tạo timed transcript với metadata đầy đủ

Kết quả

- Raw transcript JSON với segments và timestamps
- Định dạng: {`video_name`, `segments`, `model`, `created_at`}

Pipeline Xử lý Transcript ASR

1. **Chuẩn hoá Metadata**
   - Kiểm tra và chuẩn hoá metadata
   - Tách thông tin chapter/part/section từ tên video
2. **Sửa lỗi Code-Switch**
   - Sửa lỗi chuyển mã bằng phương pháp exact + fuzzy matching
   - Khôi phục dấu câu bằng *punctuation restoration* với mô hình *GecBERTModel*
   - Làm sạch văn bản (*text cleaning*) bao gồm xử lý các dạng viết tắt
3. **LLM Enhancement**
   - Rewrite bằng mô hình Arcee-VyLinh để tăng độ rõ ràng
   - Tạo *dual representation*: bản gốc + bản đã được cải thiện
   - Thực hiện *validation*: giữ nguyên ý nghĩa, kiểm tra *semantic similarity* và bảo toàn các *key terms*
4. **Pre-chunking**
   - Ghép các segment thành văn bản liên tục
   - Loại bỏ lặp lại giữa các segment
5. **Semantic Chunking**
   - Chia đoạn bằng *RollingWindowSplitter* kết hợp Vietnamese Embedding
   - Tạo các chunk với *dual representation* (cleaned + enhanced)
   - Ước lượng *timestamp* dựa trên segment gốc

Kết quả Đầu ra

Cấu trúc JSON sau khi xử lý video gồm các thành phần chính:

- **video_name**: Tên video + cấu trúc Chapter/Part/Section.
- **created_at**: Thời điểm sinh file JSON.
- **num_chunks**: Tổng số đoạn (chunks) được tách từ video.
- **source_metrics**:
  - total_chunks – tổng số đoạn
  - avg_cleaned_length – độ dài trung bình văn bản "cleaned"
  - avg_llm_length – độ dài trung bình văn bản LLM rewrite
- **chunks**: Danh sách các đoạn đã tách.
  - chunk_id – ID duy nhất của đoạn
  - start / end – thời điểm bắt đầu & kết thúc trong video
  - metadata – thông tin phân cấp (chapter, part, section...)
  - original – văn bản đã làm sạch (stage 1)
  - enhanced – văn bản đã được LLM rewrite (stage 2)
  - metrics – thông tin thống kê (số lượng từ...)

Kết quả Đầu ra


```
{
  "video_name": "CH02_P01_S01--Mô_hình_học_tổng_quát",
  "created_at": "2025-11-17T05:08:16.900142",
  "num_chunks": 28,
  "source_metrics": {
    "total_chunks": 28,
    "avg_cleaned_length": 119,
    "avg_llm_length": 111
  },
  "chunks": [
    {
      "chunk_id": "CH02_P01_S01--Mô_hình_học_tổng_quát_chunk_0",
      "start": 15.234,
      "end": 73.234,
      "metadata": {
        "chapter_id": "CH02",
        "part_id": "P01",
        "section_id": "S01",
        "chunk_index": 0,
        "total_chunks": 28
      },
      "original": "Rồi chào các bạn...",
      "enhanced": "Rồi chào các bạn...",
      "metrics": {
        "cleaned_word_count": 160,
        "llm_word_count": 146
      }
    }
  ]
```

Retrieval Augmented Generation (RAG)

- **Khái niệm:** RAG là kỹ thuật kết hợp sức mạnh giữa khả năng tìm kiếm thông tin (Retrieval) và khả năng sáng tạo văn bản (Generation) nhằm khắc phục các hạn chế về kiến thức của mô hình ngôn ngữ lớn (LLM).
- **Cơ chế hoạt động:** Thay vì phụ thuộc hoàn toàn vào dữ liệu đã được huấn luyện trước đó, hệ thống RAG sẽ chủ động truy xuất thông tin từ các nguồn dữ liệu bên ngoài (như transcript video) để làm ngữ cảnh, giúp LLM tổng hợp câu trả lời chính xác và có cơ sở thực tế (grounded).

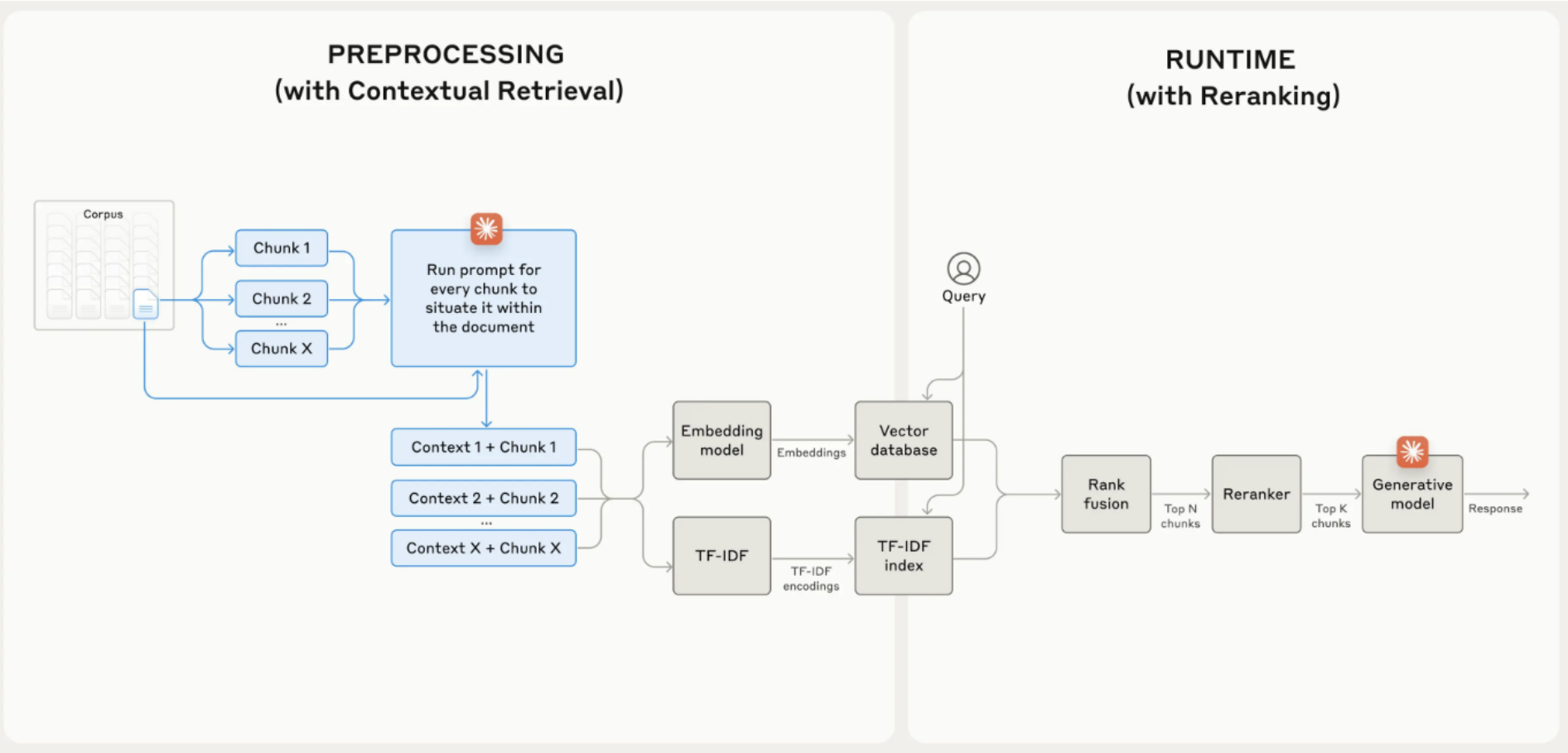
Điểm mạnh:

- Giảm thiểu đáng kể hiện tượng "ảo giác"(hallucination) nhờ việc ràng buộc câu trả lời vào dữ liệu thực tế được cung cấp.
- Nâng cao độ chính xác và tính cập nhật cho hệ thống, đặc biệt hiệu quả trong các bài toán hỏi đáp (Q&A) trên dữ liệu chuyên biệt.
- Hệ thống hoạt động linh hoạt nhờ sự kết hợp giữa tìm kiếm ngữ nghĩa (dense) và tìm kiếm từ khóa (sparse), cùng khả năng hỗ trợ đa ngôn ngữ (bao gồm tiếng Việt).

Kiến trúc hệ thống

- **Query Embedding:** Hệ thống sử dụng song song ba mô hình vector khác nhau (BGE-M3, Multilingual-E5 và Vietnamese-embedding) để mã hóa câu hỏi, đảm bảo khả năng hiểu đa ngôn ngữ và tối ưu hóa cho tiếng Việt.
- **Semantic Retrieval:** Quá trình tìm kiếm diễn ra đồng thời trên cả ba không gian vector để bao quát tối đa các sắc thái ngữ nghĩa của câu truy vấn.
- **Full-text Retrieval:** Hệ thống bổ sung thuật toán BM25 để tìm kiếm dựa trên sự trùng khớp từ khóa chính xác, giúp bắt được những thuật ngữ cụ thể mà tìm kiếm vector có thể bỏ sót.
- **Result Fusion:** Kết quả từ các nguồn tìm kiếm khác nhau sẽ được hợp nhất bằng thuật toán Reciprocal Rank Fusion (RRF) để chọn lọc ra danh sách ứng viên cân bằng nhất.
- **Reranking:** Danh sách kết quả sơ bộ sẽ được sắp xếp lại bởi mô hình Cross-Encoder nhằm đánh giá độ tương quan chi tiết và chính xác hơn.
- **Answer Generation:** Cuối cùng, mô hình ngôn ngữ (Gemini) sẽ tổng hợp thông tin và sinh câu trả lời hoàn chỉnh, kèm theo trích dẫn thời gian (timestamps) cụ thể.

Mô tả kiến trúc



Bước 2: Chiến lược truy vấn dữ liệu

Vector Search (Tìm kiếm ngữ nghĩa)

- Hệ thống thực hiện tìm kiếm kết hợp trên các embeddings của cả ba mô hình (BGE, Vietnamese, ME5) để tận dụng điểm mạnh riêng của từng loại.
- Mỗi mô hình sẽ truy vấn trên không gian vector được đánh chỉ mục riêng biệt và trả về 100 kết quả phù hợp nhất (Top-100).

BM25 Search (Tìm kiếm từ khóa)

- Song song với vector, hệ thống thực hiện tìm kiếm full-text dựa trên tần suất xuất hiện của từ khóa trong tài liệu.
- Phương pháp này đặc biệt hữu ích để tìm kiếm các từ ngữ chuyên ngành hoặc cụm từ chính xác mà Vector Search có thể vô tình bỏ qua.

Kết quả tổng hợp: Đầu ra của bước này là tập hợp các kết quả tốt nhất từ 3 nguồn vector và 1 nguồn BM25, sẵn sàng cho bước hợp nhất tiếp theo.

Bước 3: Thuật toán Reciprocal Rank Fusion (RRF)

Khái niệm: RRF là phương pháp hợp nhất thứ hạng từ nhiều danh sách tìm kiếm khác nhau để tạo ra một kết quả chung cân bằng nhất. Điểm mạnh của RRF là khả năng tận dụng ưu điểm của từng nguồn tìm kiếm (Bước 2).

$$\text{score}(d) = \sum_{q \in \text{queries}} \frac{1}{k + \text{rank}(d \mid q)}$$

(Công thức này tính điểm cho tài liệu bằng cách cộng nghịch đảo thứ hạng của nó từ các nguồn, ưu tiên những tài liệu xuất hiện ở vị trí cao trong nhiều danh sách.)

Quy trình xử lý:

- **Tính điểm:** Mỗi tài liệu sẽ được tính tổng điểm dựa trên vị trí xếp hạng của nó trong kết quả từ 4 nguồn truy vấn (3 Vector + 1 BM25).
- **Hợp nhất:** Hệ thống loại bỏ các kết quả trùng lặp và tổng hợp điểm số cuối cùng.
- **Sàng lọc:** Danh sách tài liệu được sắp xếp theo điểm số giảm dần để chọn ra 50 kết quả chất lượng nhất (Top-50) chuyển sang bước tinh chỉnh.

Bước 4: Tinh chỉnh kết quả (Cross-Encoder Reranking)

Vai trò của Cross-Encoder: Khác với các mô hình embedding chỉ so sánh vector một cách riêng lẻ, Cross-Encoder đánh giá độ tương quan bằng cách xử lý đồng thời cả câu hỏi (query) và tài liệu (document). Cơ chế này cho phép mô hình hiểu sâu hơn về ngữ cảnh để tinh chỉnh thứ tự xếp hạng, giúp tăng đáng kể độ chính xác (precision).

Quy trình thực hiện:

- **Đầu vào:** Tiếp nhận danh sách 50 ứng viên tiềm năng nhất đã được hợp nhất từ thuật toán RRF.
- **Đánh giá:** Mô hình sẽ chấm điểm chi tiết từng cặp (câu hỏi - tài liệu) dựa trên các tín hiệu ngữ nghĩa tinh tế, chẳng hạn như khả năng giải thích vấn đề thay vì chỉ nhắc đến từ khóa.
- **Đầu ra:** Chọn lọc ra 20 đoạn ngữ cảnh (contexts) có độ tin cậy cao nhất để cung cấp cho mô hình ngôn ngữ.

Bước 5: Tổng hợp câu trả lời (Generation)

Lựa chọn ngữ cảnh tối ưu:

- Từ danh sách 20 kết quả đã được tinh chỉnh, hệ thống tiếp tục chọn ra 3–5 đoạn thông tin giàu giá trị nhất để làm đầu vào cho LLM.

Xử lý và sinh văn bản:

- Hệ thống sử dụng Gemini - mô hình ngôn ngữ có khả năng xử lý tiếng Việt vượt trội - để tổng hợp nội dung từ các ngữ cảnh đã chọn.
- Nguyên tắc quan trọng nhất là câu trả lời phải được xây dựng hoàn toàn dựa trên thông tin được cung cấp, tuyệt đối không thêm thắt các dữ liệu ngoài luồng.

Trích dẫn nguồn minh bạch:

- Mỗi câu trả lời sinh ra đều đi kèm với thông tin nguồn gốc cụ thể (các đoạn chunk hoặc timestamp video), giúp người dùng dễ dàng đối chiếu và xác minh thông tin.

Đánh giá hiệu năng

Dữ liệu

Dữ liệu đánh giá của nhóm bao gồm 20 mẫu được xây dựng từ kho video bài giảng của thầy Tiệp. Mỗi mẫu dữ liệu bao gồm: câu hỏi, câu trả lời cho câu hỏi, nguồn tài liệu được sử dụng để đưa ra câu trả lời.

Example

- Câu hỏi: Vấn đề khi hàm mất mát có nhiều điểm cực tiểu (local minima) và giải pháp?
- Câu trả lời: Khi hàm mất mát có nhiều điểm cực tiểu, việc sử dụng phiên bản tối ưu hoá cơ bản có thể khiến mô hình bị kẹt tại điểm cực tiểu địa phương. Giải pháp đầu tiên là chạy thuật toán nhiều lần với các điểm khởi tạo ngẫu nhiên khác nhau và chọn ra kết quả tốt nhất. Giải pháp thứ hai là sử dụng kỹ thuật momentum (quán tính) giúp quá trình tối ưu vượt qua các điểm cực tiểu địa phương để hướng tới điểm tốt hơn
- Nguồn thông tin: CS431-Chương_2-Part_1-Mô_hình_học_tổng_quát: Segment = 28, 30, 32

Đánh giá hiệu năng

Độ đo

Để đánh giá mô hình RAG có hoạt động tốt hay không, nhóm đánh giá mô hình thông qua các độ đo sau đây:

- **Groundedness**: *"what extent does the generated response agree with the retrieved context"*. Evaluator: LLM-as-judge
- **Contextual Relevancy (CR)**: *"how relevant are my retrieved results for this query"*. Evaluator: LLM-as-judge
- **Answer Accuracy (AA)**: *"how correct is the RAG chain answer, relative to a ground-truth answer"*. Evaluator: Human
- **Source Citation Accuracy (SCA)**:

$$\text{Precision} = \frac{|S_{\text{gt}} \cap S_{\text{pred}}|}{|S_{\text{pred}}|} \qquad \text{Recall} = \frac{|S_{\text{gt}} \cap S_{\text{pred}}|}{|S_{\text{gt}}|}$$

Đánh giá hiệu năng

Bảng 1: Kết quả thực nghiệm

Metric	BGE-M3	Vietnamese Encoder	Multilingual-E5
Groundedness (0–1)	**0.9525**	0.9075	0.9200
Contextual Relevancy (0–1)	**0.8700**	0.8250	0.8250
Answer Accuracy (0–1)	**0.8350**	0.7450	0.7425
SCA Precision	**0.8384**	0.7545	0.7385
SCA Recall	**0.8864**	0.8009	0.7850

Nhận xét: Tuy mô hình cho thấy khả năng truy xuất được những thông tin quan trọng rất tốt, nhưng mô hình vẫn đôi lúc bị ảo giác và đưa ra những thông tin dư thừa, không cần thiết để trả lời câu hỏi, dẫn đến sai yêu cầu câu hỏi. Tuy nhiên vì khả năng truy xuất thông tin tốt nên hiệu suất trích nguồn tài liệu của mô hình là khá cao.

Tổng kết

Nhóm thành công xây dựng một mô hình RAG đơn giản để hỏi-đáp cũng như dẫn nguồn, giúp việc học tập trở nên dễ dàng hơn. Để chứng minh tính ứng dụng của mô hình, nhóm đã xây dựng một bộ dữ liệu nhỏ, đồng thời đánh giá mô hình trên nhiều độ đô khác nhau.